# Gestión de Pagos y Aranceles.

Guía para la explicación en clase · Checklist de requisitos y descripción de archivos · Actividad 7

## CHECKLIST DE REQUISITOS

| | REQUISITO DE LA CONSIGNA | DÓNDE ESTÁ |
|---|---|---|
| **ACTIVIDAD 7** | | |
| ✓ | Aplicación JavaFX + JPA | `App.java, persistence.xml` |
| ✓ | CRUD de conceptos (entidad Concepto: código, descripción, monto) | `Concepto, ConceptoService, vista Conceptos` |
| ✓ | CRUD de pagos (entidad Pago: estudiante, concepto, fecha, monto, estado) | `Pago, PagoService, vista Pagos` |
| ✓ | Calcular saldos pendientes | `CalculadoraSaldos, columna Saldo, árbol de cuenta` |
| ✓ | Aplicar becas | `PagoService.aplicarBeca, panel Beca` |
| ✓ | HashMap<Integer, Estudiante> | `EstudianteService (campo estudiantes)` |
| ✓ | ArrayList<Pago> | `PagoService (campo pagos)` |
| ✓ | PagoDuplicadoException | `concepto ya pagado / referencia repetida` |
| ✓ | MontoInvalidoException | `monto nulo, ≤ 0, > 2 decimales o > saldo` |
| ✓ | BecaNoAplicableException | `% fuera de (0,100], inactivo, sin saldo becable` |
| ✓ | Logger Singleton para registrar transacciones | `LoggerTransacciones → logs/transacciones.log` |
| ✓ | PagoFactory: pagos en efectivo, tarjeta o transferencia | `PagoFactory + 3 subclases de Pago` |
| ✓ | Pruebas JUnit 5: Logger, Factory y cálculo de saldos | `src/test (78 pruebas, 0 fallas)` |
| ✓ | GUI con TableView | `Pagos, Conceptos, Estudiantes` |
| ✓ | PieChart de distribución de pagos | `Resumen (por medio / por concepto)` |
| ✓ | CSS | `css/estilos.css` |
| **UNIDADES EVALUADAS (EXTRA)** | | |
| ✓ | U5: formulario Swing con eventos y Layout Manager | `swing/SimuladorBecaFrame` |
| ✓ | U6: TreeView, animaciones y MVC | `árbol de cuenta corriente, view/Animaciones, FXML + controller` |
| ✓ | U7: EntityManager, CRUD y JPQL | `dao/GenericDAO, PagoDAO (GROUP BY, JOIN FETCH)` |
| **ENTREGABLES** | | |
| ✓ | Código fuente completo (proyecto Maven) | `repositorio / .zip` |
| ✓ | Script SQL (CREATE TABLE, INSERT, UPDATE, SELECT) | `sql/pagos_aranceles.sql` |
| ✓ | Capturas de pantalla | `docs/capturas (15 imágenes)` |
| ✓ | Informe técnico (máx. 2 páginas) | `docs/Informe_Tecnico.pdf` |
| ✓ | Repositorio GitHub | `github.com/NatanielFM/SistemaDeGestionDePagosYAranceles` |

## GUION CORTO PARA LA DEMOSTRACIÓN

1. **Resumen:** cifras animadas y PieChart; alternar “Por medio” / “Por concepto” (el segundo usa una consulta JPQL con GROUP BY).
2. **Pagos:** registrar una tarjeta en 3 cuotas → aviso verde. Intentar pagar de nuevo la matrícula de Lucía → *PagoDuplicadoException*. Escribir -500 → *MontoInvalidoException* (el campo se sacude).
3. Registrar una **transferencia**: queda “Pendiente”; botón **Acreditar**. Mostrar Anular y el filtro por medio.
4. **Estudiantes:** TreeView con la cuenta corriente. Beca de 150 % → *BecaNoAplicableException*; beca de 40 % → baja el saldo.
5. **Bitácora:** cada operación anterior registrada por el Logger Singleton, también en el archivo.
6. **Simulador de becas** (Swing) y por último `mvn test`.

## QUÉ HACE CADA ARCHIVO

| ARCHIVO | PARA QUÉ SIRVE |
|---|---|
| **RAÍZ DEL PROYECTO** | |
| pom.xml | Configuración de Maven: dependencias (JavaFX, Hibernate, MySQL, H2, JUnit 5) y plugins para ejecutar. |
| sql/pagos_aranceles.sql | Crea la base y las tablas, carga datos de ejemplo y trae UPDATE y SELECT de control. |
| config/db.properties.ejemplo | Plantilla con URL, usuario y contraseña de MySQL (se copia como db.properties). |
| README.md | Presentación del proyecto y cómo ejecutarlo. |
| **COM.IPESMI.PAGOS** | |
| Main.java | Lanzador: llama a Application.launch (sirve para ejecutar fuera de Maven/NetBeans). |
| App.java | Aplicación JavaFX: carga Main.fxml y el CSS, dimensiona la ventana y cierra JPA al salir. |
| **MODEL — ENTIDADES JPA** | |
| Estudiante.java | Legajo (PK), nombre, apellido, email, % de beca y si está activo. |
| Concepto.java | Código (PK), descripción, monto, tipo, si es becable y vencimiento. |
| Pago.java | Clase abstracta: estudiante, concepto, fecha, monto, estado y referencia. Herencia SINGLE_TABLE. |
| PagoEfectivo.java | Pago en caja; la referencia es el número de recibo. |
| PagoTarjeta.java | Pago con tarjeta: últimos 4 dígitos y cantidad de cuotas. |
| PagoTransferencia.java | Transferencia: número de operación y banco de origen. |
| EstadoPago / MedioPago / TipoConcepto | Enums: Pendiente/Acreditado/Anulado, Efectivo/Tarjeta/Transferencia, Matrícula/Cuota/Examen/Otro. |
| **DAO — ACCESO A DATOS** | |
| JPAUtil.java | Crea un único EntityManagerFactory y arma la configuración (persistence.xml, db.properties, modo demo). |
| GenericDAO.java | CRUD genérico: guardar, actualizar, eliminar, buscar y listar, con transacciones. |
| EstudianteDAO / ConceptoDAO | Listados ordenados con JPQL. |
| PagoDAO.java | JPQL: pagos con JOIN FETCH, conteos y total acreditado por tipo de concepto (GROUP BY). |
| **SERVICE — LÓGICA DE NEGOCIO** | |
| EstudianteService.java | ABM de estudiantes con validaciones; HashMap<Integer, Estudiante>. |
| ConceptoService.java | ABM de conceptos y validación de montos. |
| PagoService.java | Registrar, editar, acreditar, anular y eliminar pagos; becas; saldos; datos del PieChart. ArrayList<Pago>. |
| CalculadoraSaldos.java | Fórmulas puras: importe con beca, saldo por concepto, saldo total y monto disponible. |
| AppContext.java | Crea los servicios una sola vez y los comparte entre los controladores. |
| DatosDemo.java | Carga los datos de ejemplo del script cuando se usa el modo demo (H2). |
| **FACTORY · EXCEPTION · UTIL** | |
| PagoFactory.java | Patrón Factory: crea la subclase de Pago según el medio, valida sus datos y fija el estado inicial. |
| SolicitudPago.java | Record con los datos que recibe la fábrica. |
| ArancelException.java | Base (checked) de las excepciones de negocio. |
| PagoDuplicado / MontoInvalido / BecaNoAplicable | Las tres excepciones personalizadas que pide la consigna. |
| LoggerTransacciones.java | Patrón Singleton: registra cada transacción y error con fecha y nivel en el archivo de log. |

| | |
|---|---|
| `Formatos.java` | Formato de pesos y fechas (es-AR) y lectura de montos como “45.000,50”. |
| **`CONTROLLER · VIEW · SWING (INTERFAZ)`** | |
| `MainController.java` | Barra lateral, navegación con indicador animado, conexión en segundo plano y apertura del simulador Swing. |
| `ResumenController.java` | Cifras animadas, PieChart y ranking de saldos. |
| `PagosController.java` | TableView de pagos con filtros y formulario de alta/edición; atrapa las excepciones y muestra avisos. |
| `ConceptosController.java` | TableView y formulario de conceptos. |
| `EstudiantesController.java` | TableView de estudiantes, formulario, becas y TreeView de cuenta corriente. |
| `BitacoraController.java` | Muestra el log con colores por nivel y se actualiza en vivo. |
| `Refrescable.java` | Interfaz para que cada vista recargue sus datos al mostrarse. |
| `Animaciones.java` | Transiciones reutilizables: entrada, cascada, sacudida, contador y barras. |
| `Notificador.java` | Avisos emergentes (toasts) de éxito, error e información. |
| `Celdas.java` | Celdas personalizadas de tabla (montos, etiquetas de estado, medio de pago). |
| `Dialogos.java` | Diálogo de confirmación con estilo propio (anular / eliminar). |
| `SimuladorBecaFrame.java` | Formulario Swing que calcula importe con beca y valor de cuota. |
| **`RESOURCES`** | |
| `META-INF/persistence.xml` | Unidad de persistencia “pagosPU”: entidades y conexión por defecto a MySQL. |
| `view/*.fxml` | Vistas: Main, Resumen, Pagos, Conceptos, Estudiantes y Bitácora. |
| `css/estilos.css` | Estilos de toda la aplicación (colores, tablas, botones, PieChart, avisos). |
| **`SRC/TEST`** | |
| `LoggerTransaccionesTest.java` | Unicidad (también con varios hilos), escritura en archivo y mensajes nulos/vacíos. |
| `PagoFactoryTest.java` | Crea los 3 tipos, rechaza tipos inválidos y valida dígitos, cuotas y nro. de operación. |
| `CalculadoraSaldosTest.java` | Saldos con pagos parciales, pendientes, anulados, becas 0–100 % y centavos. |
| `PagoServiceTest.java` | Servicio real con JPA sobre H2: duplicados, montos límite, becas, acreditación y JPQL. |
| `CapturasPantalla.java` | Herramienta (no es una prueba) que recorre la app y genera las capturas de docs/capturas. |